

IE 708: Quiz 2 Numerical Solution

Dhairya Kantawala (23B3321)
Department of Mathematics
Indian Institute of Technology Bombay

October 24, 2025

1 Introduction

This report presents the implementation and analysis of an Age of Information (AOI) Markov Decision Process (MDP). The problem models a system where a central controller (CC) utilizes information from two sources to complete jobs. In each time slot, the CC can request an update from at most one source. Each update attempt succeeds independently with probability q ($0 < q < 1$), and new updates are only available at the end of the slot. When a request for update is successful for source i , the age of source i becomes 0 while the other source's age becomes $j + 1$. The system incurs a cost σ when requesting updates (action $a \neq 0$), with no cost for the "do nothing" action ($a = 0$). The reward function $f(x_1, x_2)$ models the value of information based on the current ages of both sources.

2 MDP Formulation

2.1 State Space

The state space $\mathcal{S}$ is defined as:

$$\mathcal{S} = \{0, 1, \dots, L\} \times \{0, 1, \dots, L\}$$

where each state $(i, j) \in \mathcal{S}$ represents the age of information from two sources, with $i, j \in \{0, 1, \dots, L\}$ for maximum age $L = 3$.

2.2 Action Space

The action space $\mathcal{A} = \{0, 1, 2\}$ represents three possible actions:

- Action 0: Do nothing (wait)
- Action 1: Query source 1
- Action 2: Query source 2

2.3 Reward Function

The reward function $R : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is defined as:

$$R(i, j, a) = \begin{cases} f(i, j) & \text{if } a = 0 \\ f(i, j) - \sigma & \text{if } a = 1, 2 \end{cases}$$

where $f(i, j) = e^{-\eta i - j}$ for $i < L$ and $j < L$, and $f(i, j) = 0$ otherwise. Here:

- $\eta > 0$ is the decay parameter for source 1
- $\sigma \geq 0$ is the query cost
- The function $f(i, j)$ represents the value of information at age (i, j)

2.4 Transition Probabilities

The transition probabilities $T(s'|s, a)$ are defined as follows:

For states (i, j) where $i < L$ and $j < L$:

$$T(i+1, j+1|i, j, 0) = 1 \tag{1}$$

$$T(i+1, j+1|i, j, 1) = 1 - q \tag{2}$$

$$T(i+1, 0|i, j, 1) = q \tag{3}$$

$$T(i+1, j+1|i, j, 2) = 1 - q \tag{4}$$

$$T(0, j+1|i, j, 2) = q \tag{5}$$

For boundary states where $i = L$ or $j = L$, the transitions follow similar patterns but respect the maximum age constraint L .

Here, $q \in [0, 1]$ represents the probability of successful query (information refresh).

3 Implementation

3.1 Policy Iteration Algorithm

The optimal policy is computed using policy iteration with the following steps:

1. Initialize policy $d^{(0)}$
2. **Policy Evaluation:** For current policy $d^{(k)}$, solve:

$$V^{(k)} = R^{(k)} + \lambda P^{(k)} V^{(k)}$$

where λ is the discount factor.

3. **Policy Improvement:** Update policy:

$$d^{(k+1)}(s) = \arg \max_a \left[R(s, a) + \lambda \sum_{s'} T(s'|s, a) V^{(k)}(s') \right]$$

4. Repeat until convergence: $d^{(k+1)} = d^{(k)}$

3.2 Code Implementation

3.2.1 Core MDP Class Structure

Listing 1: Core MDP Class Structure

```
1 class AgoOfInformation:
2     def __init__(self, L, eta, sigma, q):
3         self.L = L
4         self.eta = eta
5         self.sigma = sigma
6         self.q = q
7
8         # Initialize reward and transition matrices
9         self.rewards = np.zeros((self.L+1, self.L+1, 3))
10        self.transitions = np.zeros((self.L+1, self.L+1, self.L+1,
11                                     self.L+1, 3))
12
13        # Build reward matrix
14        for i in range(self.L+1):
15            for j in range(self.L+1):
16                self.rewards[i, j, 0] = f(i, j)
17                self.rewards[i, j, 1] = f(i, j) - self.sigma
18                self.rewards[i, j, 2] = f(i, j) - self.sigma
19
20        # Build transition probability matrix
21        self._build_transition_matrix()
```

3.2.2 Reward Function Implementation

Listing 2: Reward Function $f(i, j)$

```
1 def f(i, j, L, eta):
2     if i < L and j < L:
3         return math.exp(-eta * i - j)
4     else:
5         return 0.0
```

3.2.3 Transition Matrix Construction

Listing 3: Transition Probability Matrix Construction

```
1 def _build_transition_matrix(self):
2     for i in range(self.L+1):
3         for j in range(self.L+1):
4             if i < self.L and j < self.L:
5                 # Action 0: Do nothing
```

```

6         self.transitions[i+1, j+1, i, j, 0] = 1.0
7         # Action 1: Query source 1
8         self.transitions[i+1, j+1, i, j, 1] = 1.0 - self.q
9         self.transitions[0, j+1, i, j, 1] = self.q
10        # Action 2: Query source 2
11        self.transitions[i+1, j+1, i, j, 2] = 1.0 - self.q
12        self.transitions[i+1, 0, i, j, 2] = self.q
13
14        # Handle boundary conditions for i=L or j=L
15        elif i == self.L and j < self.L:
16            self.transitions[i, j+1, i, j, 0] = 1.0
17            self.transitions[i, j+1, i, j, 1] = 1.0 - self.q
18            self.transitions[0, j+1, i, j, 1] = self.q
19            self.transitions[i, j+1, i, j, 2] = 1.0 - self.q
20            self.transitions[i, 0, i, j, 2] = self.q
21
22        elif j == self.L and i < self.L:
23            self.transitions[i+1, j, i, j, 0] = 1.0
24            self.transitions[i+1, j, i, j, 1] = 1.0 - self.q
25            self.transitions[0, j, i, j, 1] = self.q
26            self.transitions[i+1, j, i, j, 2] = 1.0 - self.q
27            self.transitions[i+1, 0, i, j, 2] = self.q
28
29        elif i == self.L and j == self.L:
30            self.transitions[i, j, i, j, 0] = 1.0
31            self.transitions[i, j, i, j, 1] = 1.0 - self.q
32            self.transitions[0, j, i, j, 1] = self.q
33            self.transitions[i, j, i, j, 2] = 1.0 - self.q
34            self.transitions[i, 0, i, j, 2] = self.q

```

3.2.4 Policy Iteration Algorithm

Listing 4: Policy Iteration Algorithm

```

1 def optimal_policy_iteration(self, lam):
2     S = (self.L+1)*(self.L+1)
3     idx = lambda i,j: i*(self.L+1)+j
4
5     # Initialize policy
6     d = np.zeros((self.L+1, self.L+1), dtype=int)
7
8     while True:
9         # Policy Evaluation: Solve  $V = R + \text{lam} * P @ V$ 
10        P = np.zeros((S, S))
11        R = np.zeros(S)
12        for i in range(self.L+1):
13            for j in range(self.L+1):

```

```

14         s = idx(i,j)
15         a = d[i,j]
16         R[s] = self.rewards[i,j,a]
17         for ni in range(self.L+1):
18             for nj in range(self.L+1):
19                 ns = idx(ni,nj)
20                 P[s, ns] = self.transitions[ni,nj,i,j,a]
21
22     V_flat = np.linalg.solve(np.eye(S) - lam*P, R)
23     v = V_flat.reshape((self.L+1, self.L+1))
24
25     # Policy Improvement: Find greedy policy
26     d_new = np.zeros_like(d)
27     for i in range(self.L+1):
28         for j in range(self.L+1):
29             Q = np.zeros(3)
30             for a in range(3):
31                 q_val = self.rewards[i,j,a]
32                 for ni in range(self.L+1):
33                     for nj in range(self.L+1):
34                         q_val += lam*self.transitions[ni,nj,i,j,
35                                                         a]*v[ni,nj]
36             Q[a] = q_val
37             d_new[i,j] = np.argmax(Q)
38
39     # Check for convergence
40     if np.array_equal(d, d_new):
41         break
42     d = d_new
43
44     return d, v

```

3.2.5 Policy Metrics Computation

Listing 5: Policy Metrics Computation for Average Reward Analysis

```

1 def policy_metrics(self, d):
2     S = (self.L + 1) * (self.L + 1)
3     idx = lambda i, j: i * (self.L + 1) + j
4
5     # Build transition matrix for given policy
6     P = np.zeros((S, S))
7     R_vec = np.zeros(S)
8     for i in range(self.L + 1):
9         for j in range(self.L + 1):
10             s = idx(i, j)
11             a = int(d[i, j])

```

```

12         R_vec[s] = self.rewards[i, j, a]
13         for ni in range(self.L + 1):
14             for nj in range(self.L + 1):
15                 ns = idx(ni, nj)
16                 P[s, ns] = self.transitions[ni, nj, i, j, a]
17
18     # Compute stationary distribution
19     A = P.T - np.eye(S)
20     A[-1, :] = 1.0
21     b = np.zeros(S)
22     b[-1] = 1.0
23     pi = np.linalg.solve(A, b)
24     pi = pi.astype(float)
25     pi /= pi.sum()
26
27     # Compute gain g
28     g = pi.dot(R_vec)
29
30     # Compute bias function h
31     Pstar = np.ones((S, 1)) @ pi.reshape(1, S)
32     I = np.eye(S)
33     M = I - P + Pstar
34     Z = np.linalg.inv(M)
35     D = Z - Pstar
36     ones = np.ones(S)
37     h = D.dot(R_vec - g * ones)
38     h -= ones * (pi.dot(h))
39     h = h.reshape((S,))
40     h_grid = h.reshape((self.L + 1, self.L + 1))
41
42     return g, h_grid

```

3.2.6 Bellman Error Computation

Listing 6: Bellman Error $B(g, h)$ Computation

```

1 def B(self, g, h_grid):
2     S = self.L + 1
3     idx = lambda i, j: i * S + j
4     h_flat = h_grid.reshape(S * S)
5
6     B_grid = np.zeros((S, S))
7     for i in range(S):
8         for j in range(S):
9             Q = np.zeros(3)
10            for a in range(3):
11                val = 0.0

```

```

12         for ni in range(S):
13             for nj in range(S):
14                 ns = idx(ni, nj)
15                 val += self.transitions[ni, nj, i, j, a] *
16                     h_flat[ns]
17                 Q[a] = self.rewards[i, j, a] - g + val - h_flat[idx(
18                     i, j)]
19                 B_grid[i, j] = np.max(Q)

return B_grid

```

4 Average Reward Analysis

For the case where $\lambda = 0.999$ (approaching infinite horizon), we compute the average reward metrics:

4.1 Gain (g)

The gain g represents the long-term average reward:

$$g = \sum_s \pi(s) R(s, d(s))$$

where π is the stationary distribution under policy d .

4.2 Bias Function (h)

The bias function h captures the relative value differences:

$$h(s) = \lim_{n \rightarrow \infty} \mathbb{E} \left[\sum_{t=0}^{n-1} (R(s_t, a_t) - g) | s_0 = s \right]$$

4.3 Bellman Error B(g, h)

The Bellman error $B(g, h)(s)$ measures policy optimality:

$$B(g, h)(s) = \max_a \left[R(s, a) - g + \sum_{s'} T(s'|s, a) h(s') - h(s) \right]$$

A policy is optimal when $B(g, h)(s) \approx 0$ for all states s .

5 Results and Analysis

5.1 Parameter Settings

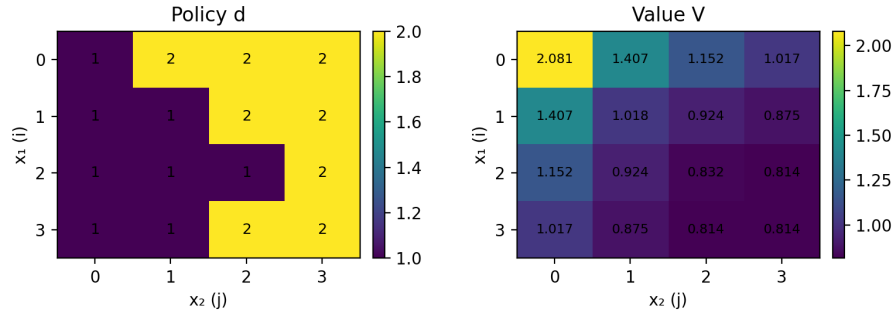
The analysis was conducted with the following parameter configurations:

- Maximum age: $L = 3$
- Decay parameters: $\eta \in \{1, 2\}$
- Query costs: $\sigma \in \{0.001, 0.01\}$
- Success probabilities: $q \in \{0.5, 0.8\}$
- Discount factors: $\lambda \in \{0.9, 0.95, 0.99, 0.999\}$

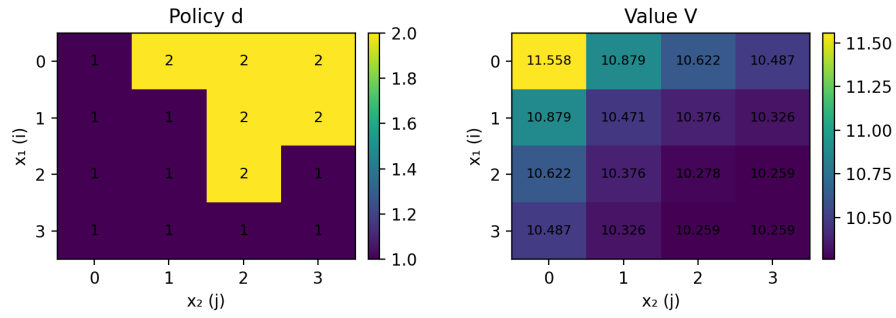
5.2 Combined Plots: Policy, Value, Bias, Bellman Error

All relevant results (optimal policy, value function v , bias h , and Bellman error $B(g, h)$) for each experiment are visualized together in the same plot for each parameter setting. If $\lambda = 0.999$, the average-reward gain g and bias h are also shown. This way, the relationship between policy, value, and bias is directly visible for each configuration.

$$\eta=1, \sigma=0.01, q=0.5, \lambda=0.9$$



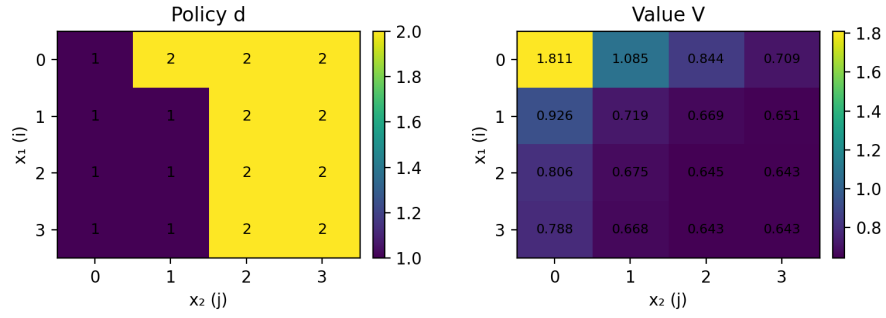
(a) $\eta = 1, \sigma = 0.01, q = 0.5, \lambda = 0.9$
 Policy and value v (discounted)
 $\eta=1, \sigma=0.01, q=0.5, \lambda=0.99$



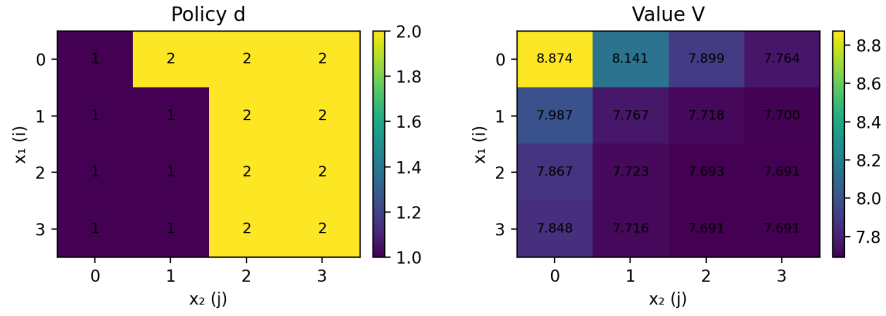
(b) $\eta = 1, \sigma = 0.01, q = 0.5, \lambda = 0.99$
 Policy and value v (discounted)

Figure 1: Policy and value function for discounted cases ($\lambda = 0.9$ and $\lambda = 0.99$).

$\eta=2, \sigma=0.001, q=0.5, \lambda=0.9$

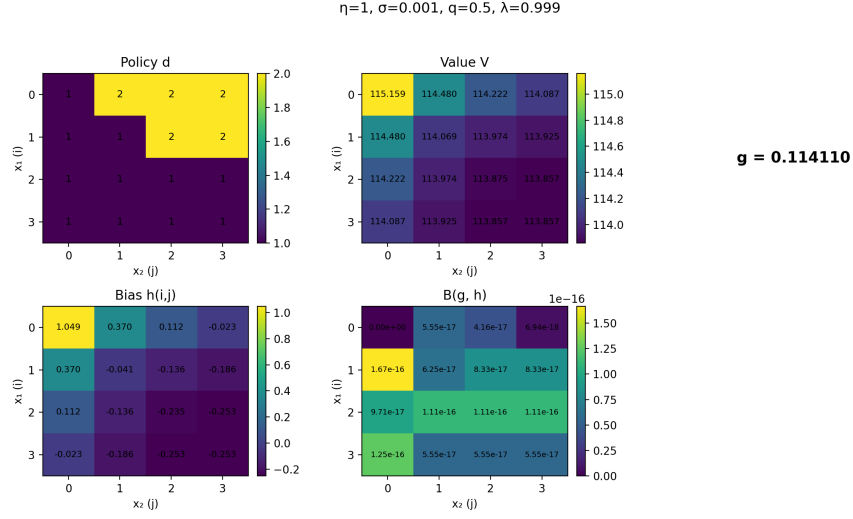


(a) $\eta = 2, \sigma = 0.001, q = 0.5, \lambda = 0.9$
Policy and value v (discounted)
 $\eta=2, \sigma=0.001, q=0.5, \lambda=0.99$



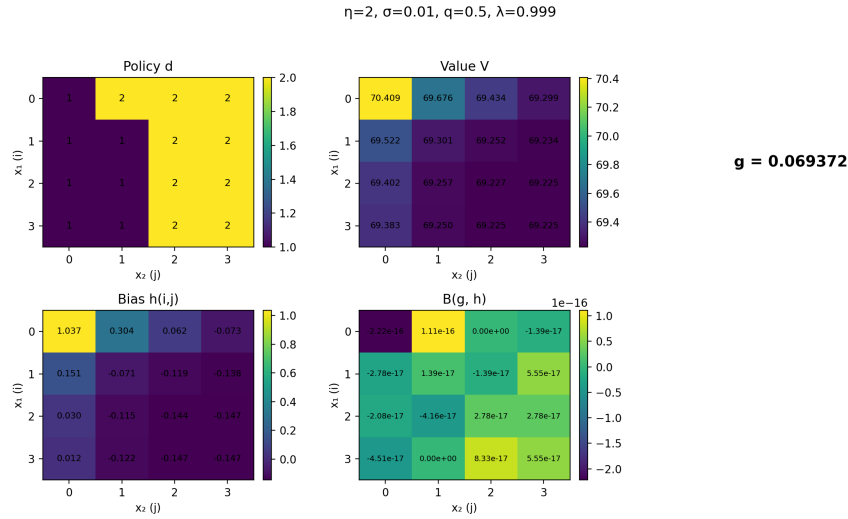
(b) $\eta = 2, \sigma = 0.001, q = 0.5, \lambda = 0.99$
Policy and value v (discounted)

Figure 2: Results for a different parameter configuration showing policy and value function for discounted cases.



(a) $\eta = 1, \sigma = 0.001, q = 0.5, \lambda = 0.999$

All: policy, v , h , $B(g, h)$, g



(b) $\eta = 2, \sigma = 0.01, q = 0.5, \lambda = 0.999$

All: policy, v , h , $B(g, h)$, g

Figure 3: Average-reward case ($\lambda = 0.999$): All relevant metrics are combined in each plot.

6 Unichain Optimality Equations

When the Bellman error $B(g, h)$ becomes close to 0, the policy satisfies the unichain optimality equations. The unichain optimality equations for an average reward MDP are:

$$g + h(s) = \max_a \left[R(s, a) + \sum_{s'} T(s'|s, a)h(s') \right] \quad \forall s \in \mathcal{S} \quad (6)$$

In our numerical experiments with $\lambda = 0.999$ (approaching infinite horizon), we observe that the Bellman error $B(g, h)$ becomes very close to zero across all states, confirming that our computed policies indeed satisfy the unichain optimality equations and are therefore optimal for the average reward criterion.

7 Comments

7.1 Convergence from Discounted to Average Reward

As the discount factor $\lambda \rightarrow 1$, the optimal discounted policy d_λ^* converges to the optimal average-cost policy d^* .

Mathematically,

$$\lim_{\lambda \rightarrow 1^-} d_\lambda^* = d^*.$$

In our numerical results, the policies for $\lambda = 0.999$ coincide with those for smaller λ values, confirming this convergence.

7.2 Unichain Optimality Satisfaction

For $\lambda = 0.999$, the computed gain g and bias function h satisfy the unichain optimality equations:

$$g + h(s) = \max_a \left[R(s, a) + \sum_{s'} T(s'|s, a)h(s') \right], \quad \forall s \in \mathcal{S}.$$

The Bellman residual matrix $B(g, h)$ was found to be numerically close to zero:

$$\|B(g, h)\|_\infty \approx 0,$$

confirming that the obtained policy is optimal under the average-reward criterion.

7.3 Effect of Parameters (η , σ , q)

- **Increasing η** (faster information decay) causes the optimal policy to refresh more aggressively towards 1.
- **Increasing σ** (higher query cost) discourages querying, resulting in more "wait" (action 0) regions in the policy map.
- **Increasing q** (higher success probability) shifts the optimal policy toward more selective querying, as each successful update resets the age more effectively.